

학생용 Workbook — 업그레이드 검증 및 문제 해결

ROS 2 Humble · LAB-11~15 · Mock LiDAR 기반 실습

이 실습은 슬라이드 11~15 와 1:1 로 연결됩니다. 학생은 소스 코드를 직접 작성하지 않습니다. 강사가 제공한 패키지와 스크립트를 실행하고, ROS 2/Git 명령으로 Evidence 를 확인한 뒤 판단과 기록을 수행합니다.

전체 실습 흐름

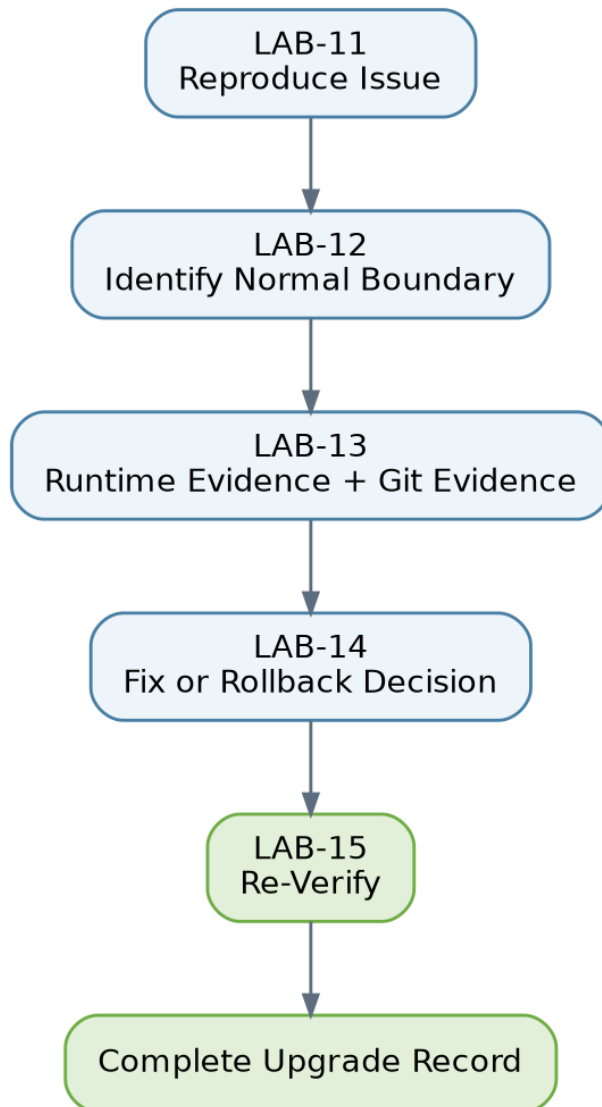


Figure 1. Workflow diagram (labels translated to English)

사전 준비

터미널에서 패키지 루트로 이동한 뒤 다음을 수행합니다.

```
./scripts/setup_lab.sh
./scripts/prepare_git_history.sh
./scripts/reset_to_v13.sh
```

prepare_git_history.sh 는 강사가 배포 직후 한 번 실행하도록 운영할 수 있습니다. 이미 v1.2, v1.3 태그가 있으면 다시 실행하지 않습니다.

LAB-11 — 업그레이드 후 센서 데이터가 사라졌다

연계 슬라이드 11 — 시연 사례

목표

- 깨진 v1.3 상태를 실행한다.
- Build/Node 실행과 기능 정상은 다를 수 있음을 확인한다.
- 이 단계에서는 원인을 추측하지 않고 **증상만 기록**한다.



Figure 2. Workflow diagram (labels translated to English)

1. 실습 상태 초기화

```
./scripts/reset_to_v13.sh
```

2. 실습 실행

워크스페이스가 빌드되어 있고 source 된 터미널에서 다음을 실행합니다.

```
ros2 launch maintenance_lidar_demo lab_current.launch.py
```

3. 관찰

약 1 초 이후 다음 의미의 경고가 나타나는지 확인합니다.

```
Laser scan data timeout
```

4. 기록

항목	관찰 결과
/lidar_node 실행	
/navigation_scan_monitor 실행	
경고 발생	
증상 요약	

이 단계에서는 원인을 작성하지 않습니다.

LAB-12 — Step 1: 어디까지 정상인지 확인한다

연계 슬라이드 12 — 정상/비정상 경계 찾기

목표

Node → Topic → Publisher → Data 순서로 확인하여 문제 범위를 좁힙니다.

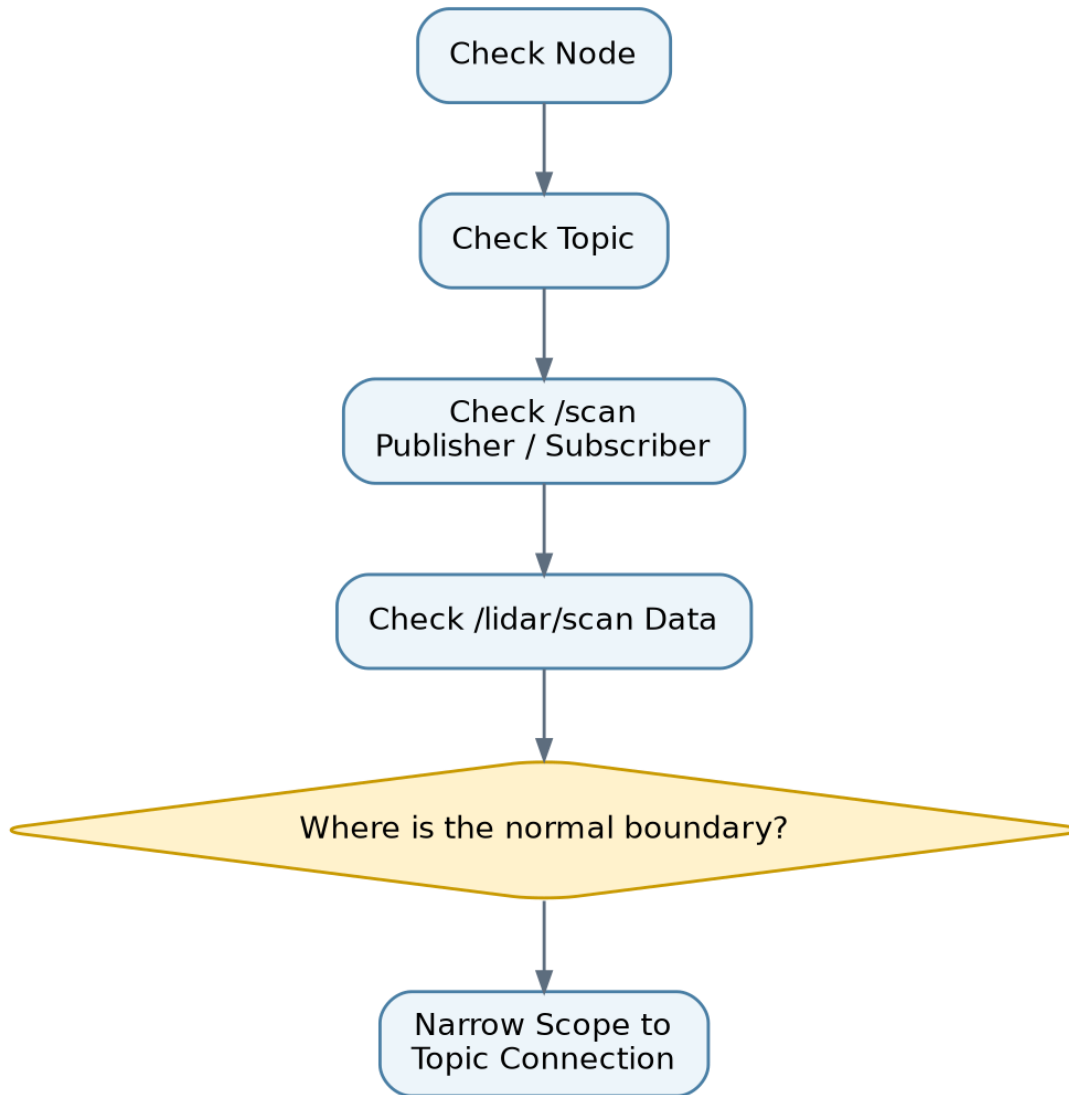


Figure 3. Workflow diagram (labels translated to English)

LAB-11 launch 는 실행 상태로 두고, 새 터미널을 열어 다음 명령을 수행합니다.

1. Node 확인

```
ros2 node list
```

예상 핵심 결과:

```
/lidar_node
```

```
/navigation_scan_monitor
```

2. Topic 목록 확인

```
ros2 topic list
```

/scan 과 /lidar/scan 의 존재 여부를 확인합니다.

3. 기존 /scan 연결 상태 확인

```
ros2 topic info /scan -v
```

기록:

항목	값
Publisher count	
Subscription count	

4. 새 Topic 의 실제 Data 확인

```
ros2 topic echo /lidar/scan --once
```

header, frame_id, ranges 가 출력되면 실제 LaserScan Data 가 생성되고 있는 것입니다.

5. 판단

다음 표를 채웁니다.

확인 대상	정상/이상	근거
Sensor Node		
/scan 연결		
/lidar/scan Data		
현재 문제 범위		

권장 판단 형식: “센서 전체”가 아니라 “Topic 연결 영역”까지 문제 범위를 좁혔다.

LAB-13 — Step 2: 변경 전후의 차이에서 원인을 찾는다

연계 슬라이드 13 — Runtime Evidence 와 Git Evidence 연결

목표

ROS 2 에서 확인한 실행 상태와 Git 변경 이력을 연결하여 Root Cause 를 설명합니다.

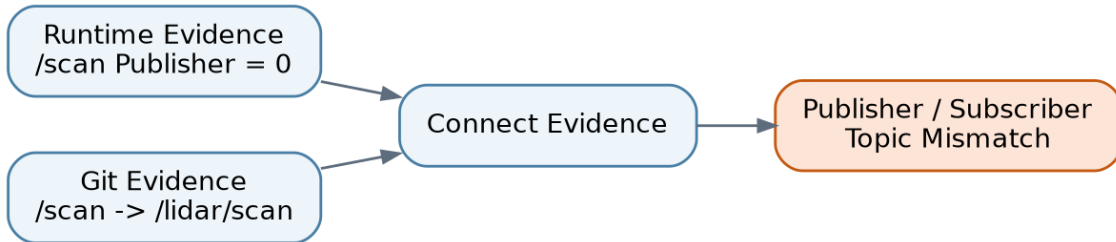


Figure 4. Workflow diagram (labels translated to English)

1. Git 이력 확인

```
git log --oneline --decorate --graph -5
```

v1.2, v1.3 태그를 확인합니다.

2. 업그레이드 전후 비교

```
git diff v1.2..v1.3
```

다음 변경을 찾습니다.

```
- scan_topic: /scan
+ scan_topic: /lidar/scan
```

3. Evidence 연결

Evidence	관찰 결과
Runtime Evidence — /scan Publisher	
Runtime Evidence — /lidar/scan Data	
Git Evidence — 변경된 Topic	
Root Cause	

최종 판단은 **Publisher Topic** 과 **Subscriber Topic** 의 불일치를 근거와 함께 작성합니다.

LAB-14 — Step 3: 수정할 것인가, Rollback 할 것인가?

연계 슬라이드 14 — Fix / Rollback 판단

목표

원인이 확인된 뒤 “고칠 수 있는가?”가 아니라 “안전하게 고칠 수 있는가?”를 기준으로 판단합니다.

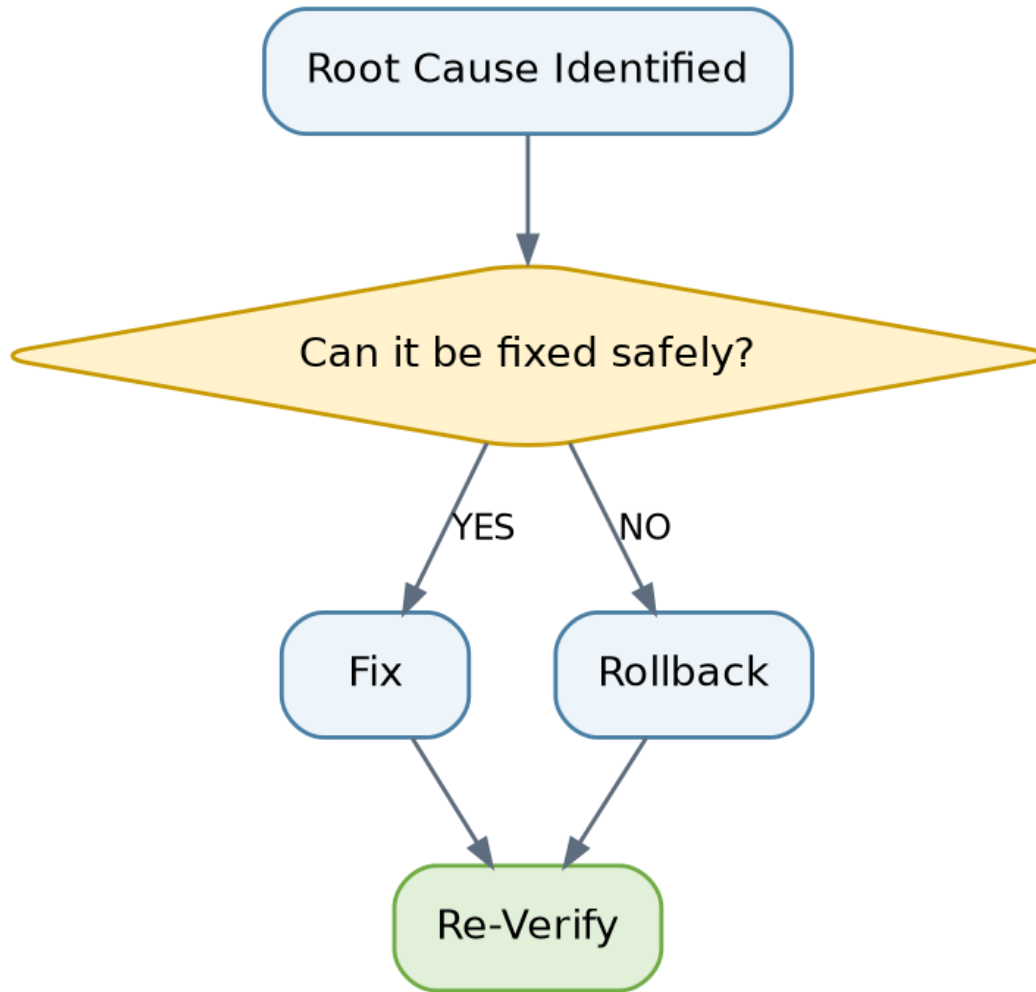


Figure 5. Workflow diagram (labels translated to English)

1. 판단표 작성

판단 기준	YES/NO	근거
원인이 명확한가?		
변경 범위가 작은가?		
설정 수정으로 해결 가능한가?		
영향 범위를 예측 가능한가?		

이번 대표 시나리오의 기본 판단은 **Fix** 입니다.

2. Fix 적용

LAB-11 launch 를 Ctrl+C 로 종료한 후 다음을 실행합니다.

```
./scripts/apply_topic_fix.sh
```

학생이 YAML/Python 소스를 직접 수정하지 않고 강사가 제공한 스크립트를 사용합니다.

3. Rollback 대안 확인

Rollback 은 실패가 아니라 복구 전략입니다. 강사 지시에 따라 대안 경로를 체험할 경우 다음을 사용합니다.

```
./scripts/rollback_v12.sh
```

Primary 실습은 다시 `./scripts/reset_to_v13.sh` 후 Fix 를 적용하여 **v1.3 유지** 상태로 진행합니다.

LAB-15 — Step 4: 수정했다고 끝내지 말고 다시 검증한다

연계 슬라이드 15 — Re-Verify & Upgrade Record

목표

수정 후 처음과 같은 기준으로 Node, Topic, Data, Frequency 를 다시 확인하고 결과를 Upgrade Record 에 기록합니다.



Figure 6. Workflow diagram (labels translated to English)

1. 수정된 상태 실행

```
ros2 launch maintenance_lidar_demo lab_current.launch.py
```

새 터미널에서 검증합니다.

2. Node 재검증

```
ros2 node list
```

3. Topic 연결 재검증

```
ros2 topic info /lidar/scan -v
```

예상 핵심:

```
Publisher count: 1
Subscription count: 1
```

4. Data 재검증

```
ros2 topic echo /lidar/scan --once
```

5. Frequency 재검증

```
ros2 topic hz /lidar/scan
```

몇 초간 관찰한 뒤 Ctrl+C 로 종료합니다. 목표는 약 10 Hz 입니다.

6. 최종 판단

확인 항목	Baseline	문제 발생	조치 후
Node	정상	정상	
Topic 연결	정상	불일치	
Data	정상	Navigation 미수신	
Frequency	약 10 Hz	기존 /scan 측정 불가	

확인 항목	Baseline	문제 발생	조치 후
Navigation Monitor	정상	Timeout	

7. Upgrade Record 작성

docs/03_upgrade_record.md 를 열어 다음을 기록합니다.

- 발생 문제
- Root Cause
- 조치 내용
- 재검증 결과
- 최종 상태

최종 상태는 Evidence 를 근거로 기록하며, 단순히 “수정했다”가 아니라 같은 기준으로 재검증했다는 점을 포함합니다.

전체 실습 종료 체크

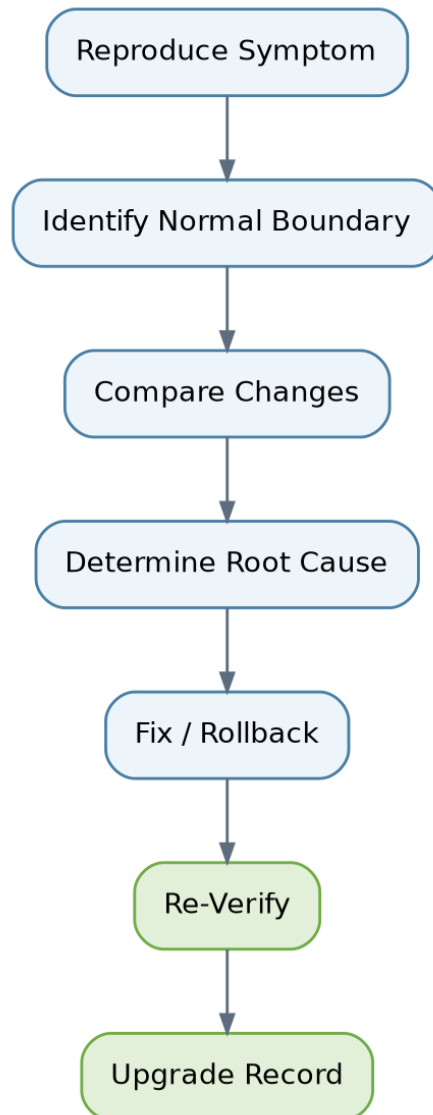


Figure 7. Workflow diagram (labels translated to English)

- ☐ LAB-11 증상을 기록했다.
- ☐ LAB-12 문제 범위를 좁혔다.
- ☐ LAB-13 Runtime Evidence 와 Git Evidence 를 연결했다.
- ☐ LAB-14 Fix/Rollback 판단 근거를 작성했다.
- ☐ LAB-15 동일 기준으로 재검증하고 Upgrade Record 를 작성했다.